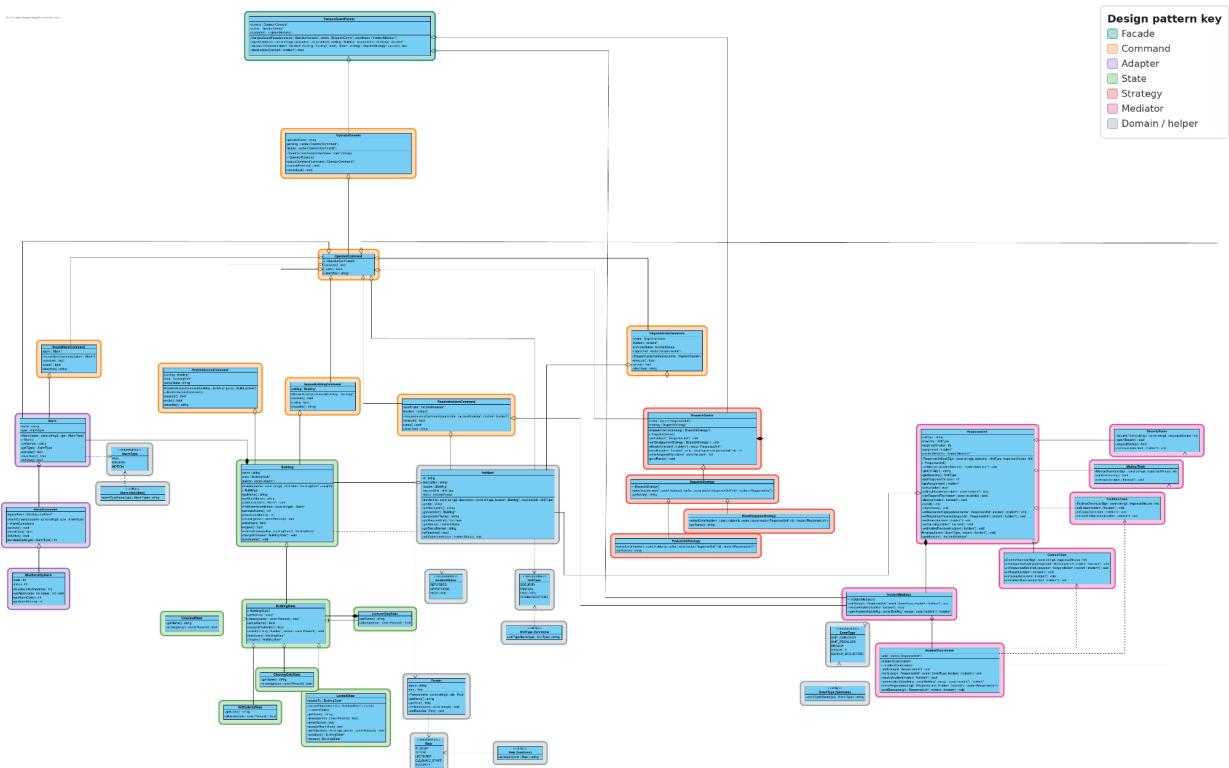


COS 214 - Practical 5

System overview

CampusGuard's job is to coordinate emergency responses on any sort of campus. Operators report incidents, dispatch response units, control building access and sound alarms. Response teams coordinate through a central incident coordinator.

UML



Team Details:

Ngele Mutwanamba - u25039769

Nathan Spavins - u25054245

Tamlyn Khan - u24675815

Pattern Description

Command

Operator actions are commands. Each action, such as dispatching units, locking a building, restricting access, sounding an alarm or resolving an incident, is its own command object. The operator console queues the commands and executes them. It can also cancel the last command, which undoes it. Each command calls the part of the system that does the real work, such as the dispatch centre, a building, an alarm or the incident coordinator.

GoF Role	Class
Client	main
Command	OperatorCommand
Concrete Command	DispatchUnitsCommand, SecureBuildingCommand, RestrictAccessCommand, SoundAlarmCommand, ResolveIncidentCommand
Invoker	OperatorConsole
Receiver	DispatchCentre, Building, Alarm, IncidentMediator

Adapter

This is the alarm system for all the buildings. It takes a legacy system that we made called blueSecurityAlarm and makes it useable for our system's alarm class Alarm. It does this by translating through the adapter class called AlarmConnector that adapts.

GoF Role	Class
Target	Alarm
Adapter	AlarmConnector

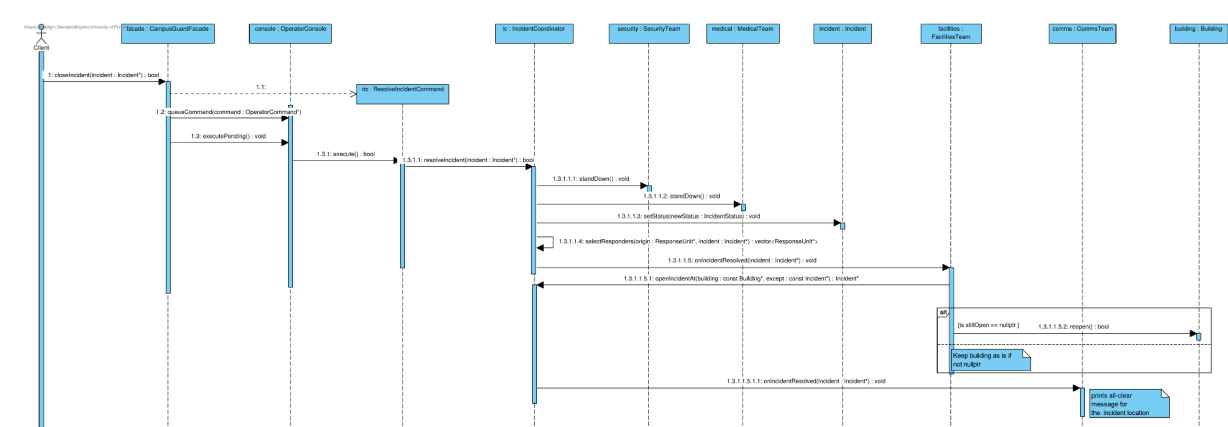
Adaptee	BlueSecurityAlarm
Client	Building

Facade

Facade turns a multi-step process into one call. It gives it one method call , `respondToBreach()`. Facade builds three Command objects, it dispatches the units, sounds the alarm and secures the building, queues them onto `OperatorConsole` and then the Console itself will run them. `reportIncident` just creates a new incident and then `closeIncident()` builds a `ResolveIncidentCommand`, queues it and executes it.

GoF Role	Class
Facade	CampusGuardFacade
Subsystem classes	DispatchCentre, Building, Alarm, IncidentMediator

Sequence Diagram:



State

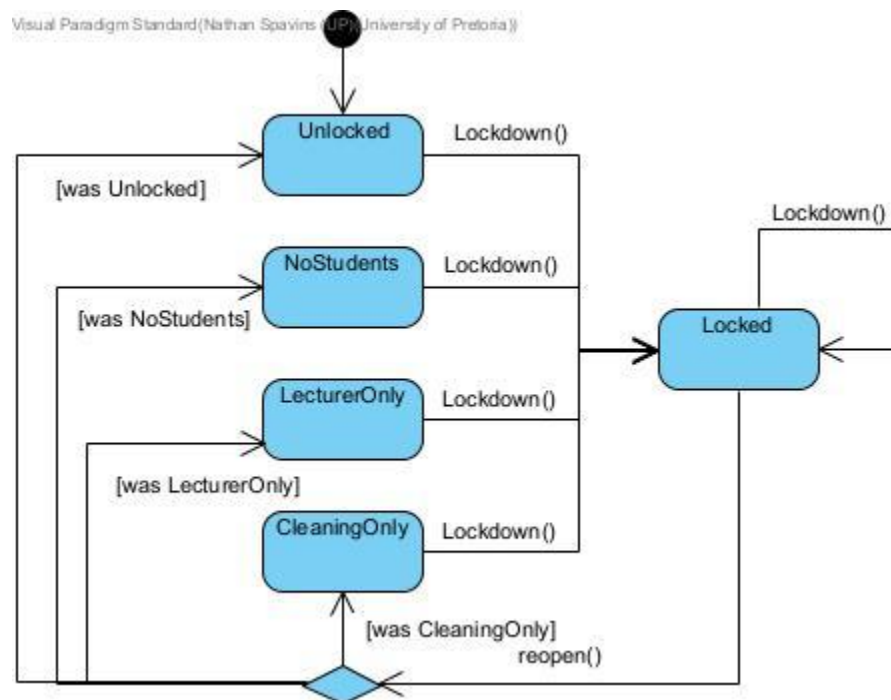
State determines the behaviour of all the buildings whether they are locked, unlocked or letting certain people in like a janitor, security, staff, student, etc. It has an alarm and provides functions to access the alarm’s utilities.

This is one of the additional patterns we added. The reason why we added it was because in a CampusGuard we felt like different buildings would be locked or unlocked or locked for certain

people. Because of this it changes behaviour. And the state pattern allows the object (Building) to alter its behaviour when its internal state changes. Therefore we chose state

GoF Role	Class
Context	Building
State	BuildingState
ConcreteStates	UnlockedState, LockedState, NoStudentState, LecturerOnlyState, CleaningOnlyState

State Diagram:



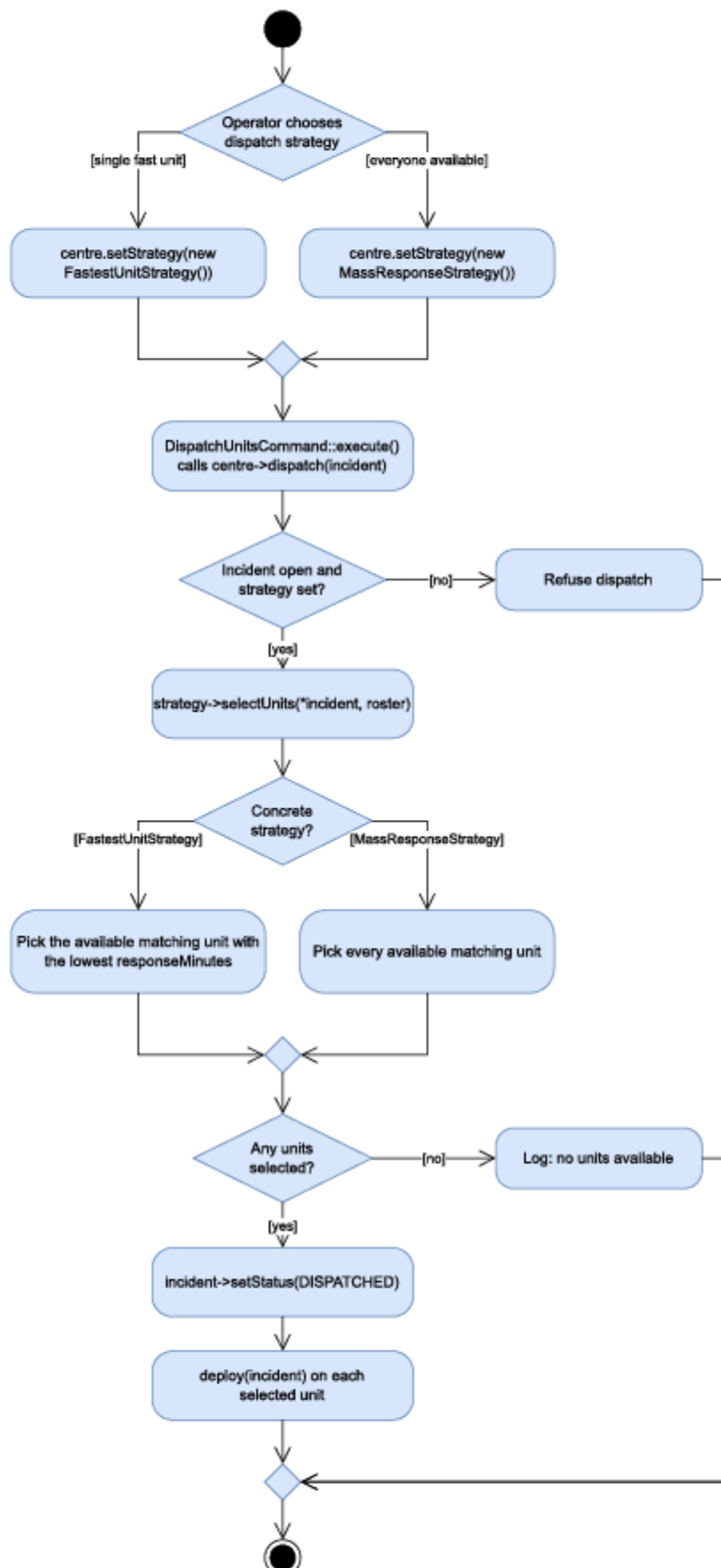
Strategy

The strategy decides which units the dispatch centre sends to an incident. The Fastest Unit strategy sends the single quickest unit of the required type. The Mass Response strategy sends every available unit of that type. The strategy can be changed at runtime, so the dispatch centre doesn't need an if/else for each way of dispatching.

GoF Role	Class
----------	-------

Context	DispatchCentre
Client	main
Strategy	DispatchStrategy
Concrete Strategy	FastestUnitStrategy, MassResponseStrategy

Activity Diagram:



Mediator

The IncidentMediator lets the 4 response teams (security, medical, facilities and comms) react to each other without knowing about each other. When something happens to the unit or team, it calls the changed() function on itself which calls notify() on the Mediator to let it know. The mediator will decide which teams need to react and call the specific method on them directly for instance onBreach() or onCausalty() etc..

GoF Role	Class
Mediator	IncidentMediator
ConcreteMediator	IncidentCoordinator
Colleague	ResponseUnit
ConcreteColleague	SecurityTeam, MedicalTeam, FacilitiesTeam, CommsTeam

Justification of our selected Patterns

State: A building's access rules change with its mode, for example Locked or Lecturer Only. Each mode is its own class, so Building needs no if/else chains, and adding a mode means adding one class.

Strategy: Different incidents need different dispatch rules, such as the fastest unit or a mass response. The rule can be swapped at runtime, so DispatchCentre never has to change when a rule is added.

Design and Ownership Rationale

Design: Each pattern handles one part of the system. Operator actions are commands. Response teams communicate only through the mediator. Legacy alarms are used through an adapter. Buildings change behaviour through their state, and the dispatch centre chooses units through a strategy.

Ownership:

Owner	Owns (deletes)	Holds without owning
-------	----------------	----------------------

DispatchCentre	its response units and current strategy (the old strategy is deleted when replaced)	N/A
IncidentCoordinator	nothing	the response units, to pass events between them
Building	its alarms and current state	N/A
LockedState	the state it replaced, returned when the building reopens	N/A
AlarmConnector	its BlueSecurityAlarm	N/A
OperatorConsole	all queued and executed commands (failed and cancelled commands are deleted immediately)	N/A
RestrictAccessCommand	the new access policy, then the old one (for undo)	its Building
Incident	nothing	its Building
ResponseUnit	nothing	its current incident and the mediator

Docker setup

Dockerfile

FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update \

&& apt-get install -y --no-install-recommends g++ make valgrind gdb \

&& rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY . .

RUN make clean && make

CMD ["/campusguard"]

docker-compose.yml

services:

campusguard:

```
build: .
image: campusguard
tty: true
cap_add:
  - SYS_PTRACE
security_opt:
  - seccomp:unconfined
```

The image installs the compiler, Valgrind and GDB, then builds the project with make. The compose file gives the container SYS_PTRACE so GDB can run inside it.

Run

```
docker compose up --build
```

Docker builds the image. The Dockerfile installs g++, make, valgrind and gdb, then runs make. After that the container starts and runs both scenarios. The: the break-in at Engineering 1, then the casualty in the Exam Hall.

Ctrl+C to exit.

GDB Investigation

Bug: After a break-in at the Lab was resolved, the Lab stayed Locked. No error was printed. This happened when the facilities unit was busy at an incident in another building.

Investigation: We set a breakpoint on FacilitiesTeam::onIncidentResolved, the only method that reopens a building, and a second one in IncidentCoordinator::resolveIncident, just before it notifies the selected responders.

```
tamly@BumbleBee:~/cos214/COS214_PA5$ cd ~/gdb-repro
gdb -q ./repro
Reading symbols from ./repro...
(gdb) break FacilitiesTeam::onIncidentResolved
Breakpoint 1 at 0x10695: file ResponseUnit.cpp, line 181.
(gdb) break IncidentCoordinator.cpp:99
Breakpoint 2 at 0xa58c: file IncidentCoordinator.cpp, line 99.
(gdb) run
Starting program: /home/tamly/gdb-repro/repro
```

```

(gdb) print incident->id
$1 = "INC-2"
(gdb) print responders.size()
$2 = 2
(gdb) print responders[0]->callSign
$3 = "SEC-1"
(gdb) print units.size()
$4 = 3
(gdb) print units[1]->callSign
$5 = "FAC-1"
(gdb) print units[1]->assignment->id
$6 = "INC-1"
(gdb) print units[1]->isAvailable()
$7 = false
(gdb) print units[1]->assignment == incident
$8 = false
(gdb) continue
A syntax error in expression, near the end of
'.
(gdb) continue
Continuing.
[COM-1] sending an all-clear for Lab
Lab after resolve: Locked
[Inferior 1 (process 32417) exited normally]
(gdb) quit

```

The first breakpoint was never hit. Only 2 of the 3 units were selected, and FAC-1 was left out.

Root cause: selectResponders only picked a unit that was available or already on this incident. FAC-1 was busy elsewhere, so the mediator never notified facilities and the Lab was never reopened.

Fix: selectResponders now picks one unit per specialty, even if that unit is busy elsewhere. After the fix, the Lab returns to Unlocked when the incident is resolved.

Valgrind Evidence

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./campusguard
```

```
CampusGuard shutting down
==6786==
==6786== HEAP SUMMARY:
==6786==      in use at exit: 0 bytes in 0 blocks
==6786==    total heap usage: 157 allocs, 157 frees, 81,704 bytes allocated
==6786==
==6786== All heap blocks were freed -- no leaks are possible
==6786==
==6786== For lists of detected and suppressed errors, rerun with: -s
==6786== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

GitHub and Collaboration Reflection

Repository: https://github.com/TamlynKhan/COS214_PA5

Each member built their pattern on a separate branch and merged it through a pull request. The main challenge was integrating the separate parts, which also exposed the bug covered in the GDB investigation.

Contributions

Tamlyn Khan: Strategy, Command, integration, Docker, Valgrind, GDB, Activity Strategy Diagram and pdf set up.

Nathan Spavins: State, Adapter, State diagram and Main.cpp.

Ngele Mutwanamba : Mediator, Facade, UML and Facade Sequence Diagram .

Commits over time

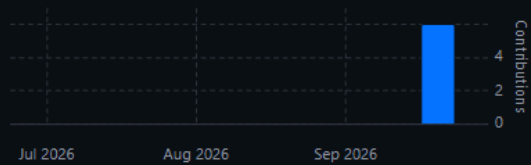
Weekly from Jun 27, 2026 to Sep 27, 2026



 **ngeletech**

6 commits 457 ++ 54 --

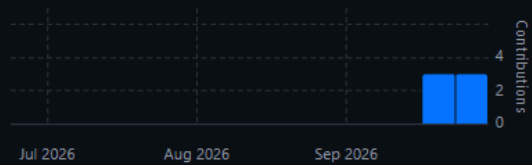
#1 ... ⚙



 **u25054245**

6 commits 615 ++ 138 --

#2 ... ⚙



 **TamlynKhan**

4 commits 2,682 ++ 510 --

#3 ... ⚙

